

Contents

Guide 3 — The Search Engine (token parser + SQLite FTS5), the simple version	1
1. The idea, in three sentences	1
2. How it works — five steps	1
3. Why FTS5 and BM25 — the part that is easy to miss	1
4. Where each part comes from	2
5. A worked example	2
6. The files	3
7. Jury questions — ready answers	3

Guide 3 — The Search Engine (token parser + SQLite FTS5), the simple version

How Mobile Match Algeria turns a free-text search like “*djezzy 5gb unlimited prepaid streaming*” into the right plans, in plain words, with a worked example and ready answers for the jury.

1. The idea, in three sentences

The search bar lets the user type the way they think, in any of the three languages. The engine splits the query into two parts: the words it **recognises** (operator, data, price, network, plan type, unlimited) become **exact database filters**, and the **leftover words** go to a small **full-text index** that ranks plans by relevance. It then shows the user, as chips, exactly how the query was understood, so nothing is a black box.

2. How it works — five steps

1. **Read the query:** lowercase it and scan it with a few small patterns (regexes).
2. **Pull out the structured tokens:** operator (djezzy), data (5gb), price (300da), network (5g), plan type (prepaid), and the keyword unlimited.
3. **Turn the tokens into a database filter:** “djezzy” fixes the operator, “5 gb” means *about 5 GB* (or unlimited), “300 da” means *around 300 DA*, “5g” the network, “prepaid” the type, “unlimited” means unlimited calls or SMS.
4. **Send the leftover words to full-text search:** any word left over (here, “streaming”) is matched against an indexed copy of every plan’s name and features, **ranked by relevance** (BM25, the same scoring search engines use).
5. **Keep the plans that pass both** the structured filter **and** the text match, best match first, and show the parsed tokens as “**Read as**” chips.

3. Why FTS5 and BM25 — the part that is easy to miss

A query has **two kinds of words**, and each kind needs a different tool.

Words the engine recognises (djezzy, 5gb, 300da, 5g, prepaid, unlimited) become exact database filters via the token parser. This is the part you already understand: operator, data, price, network, type.

Words the engine does NOT recognise (youtube, streaming, family, student, social...) are not operators or numbers, but they often sit **inside a plan’s name or its features** (for example a feature line like “Free YouTube” or “unlimited social media”). A database filter cannot match those words. That is exactly what FTS5 and BM25 are for:

- **FTS5 is the finder.** FTS5 is SQLite’s built-in **full-text search**. At scrape time we build a small **index** of every plan’s name and features, like the index at the back of a book: *word* → *the plans that contain it*. So when a leftover word like “youtube” arrives, FTS5 returns the matching plans **instantly**, instead of scanning every row

letter by letter. It also does **prefix matching** (“stream” finds “streaming”) and **accent folding** (“donnees” finds “données”).

- **BM25 is the sorter.** When several plans contain the word, which comes first? **BM25** is a standard relevance score (the same family the big search engines use) that FTS5 computes for us. It ranks a plan higher when the word is **rarer** and when it appears in the **short name** rather than buried in a long features list, so the best match ends up on top.

In one line: **the parser handles the words it knows as filters; FTS5 finds the plans that contain the other words, and BM25 puts the best match first.** Example: typing youtube matches nothing in the parser, so it goes to FTS5, which finds every plan whose features mention YouTube and lists the most relevant first.

The card shows why a plan matched. A keyword can sit in a feature the card does not normally print (a long line, or an Arabic one such as “YouTube محود...”). So when the search has a free-text word, each result card highlights the matching feature in a small accent chip (e.g. ...**YouTube**...). The user never has to wonder why a plan came up.

The search bar is “fuzzy” on purpose: “5 gb” matches *roughly* 5 GB and “300 da” matches *around* 300 DA, because a user rarely knows the exact catalogue value. The **filter panel** (separate sliders) is the place for strict minimum-data / maximum-price control.

4. Where each part comes from

Nothing here is home-made scoring:

Part of the engine	What it does	Comes from
Token / faceted parse	recognised words become exact filters	the standard structured (faceted) product-search pattern (thesis Ch. 1 §1.4)
Full-text index	searches names and features for leftover words	SQLite FTS5 , the database’s built-in full-text engine
Relevance ranking	orders the matches best-first	BM25 (Okapi BM25), the relevance function built into FTS5

What is *ours* is the configuration only: which words count as structured tokens, that “data” means *about* this much and “price” means *around* this much, and that the leftover text is ranked by FTS5. The scoring itself is BM25, computed by SQLite.

5. A worked example

Query typed by the user: djezzy 5gb unlimited prepaid streaming

Step 1 — the parser pulls out the tokens and leaves the rest as text:

Recognised	Token	Leftover text
djezzy 5gb	operator = Djezzy data ≈ 5 GB	

Recognised	Token	Leftover text
unlimited	unlimited calls or SMS	
prepaid	type = Prepaid	
streaming		→ “streaming”

Step 2 — the tokens become a database filter (Prisma where):

- operator = Djazzy
- data is unlimited **or** between 3.5 and 15 GB (the “about 5 GB” window)
- type = Prepaid
- calls **or** SMS unlimited

Step 3 — the leftover word goes to full-text search: streaming* is matched against the FTS5 index of plan names and features and ranked by BM25.

Step 4 — keep the overlap. The result is the set of **Djazzy prepaid plans of roughly 5 GB (or unlimited) with unlimited calls or SMS whose features mention streaming**, with the most relevant first.

Step 5 — the UI shows the chips: [Djazzy] [≈ 5 GB] [Unlimited] [Prepaid] [streaming], so the user can see and remove any part of how the query was read.

6. The files

- [search-tokens.ts](#) — the parser that produces the display chips.
- [api/offers/route.ts](#) — the API: the same patterns build the database filter and call the full-text search.
- [search-fts.ts](#) — builds the FTS5 index and runs the BM25 query.
- [offers/page.tsx](#) — the search bar and the “Read as” chips.

7. Jury questions — ready answers

- **Why not one big LIKE query?** A LIKE cannot rank results and is slow on long text. Splitting the query gives **exact, fast filters** for the recognised parts and a **relevance-ranked** full-text match for the words, so the result is both precise and flexible.
- **Is the ranking invented?** No. The ordering is **BM25**, the relevance function built into SQLite FTS5, the same family of scoring used by industrial search engines. We do not compute a score ourselves.
- **What if the query has no recognised words?** Then the whole query is treated as free text and handled entirely by the full-text search.
- **Why “about 5 GB” and not exactly 5 GB?** Users rarely know the exact catalogue value, so the search matches intent with a window; the filter panel offers strict minimum-data / maximum-price sliders when exact control is wanted.
- **Does it work in French and Arabic?** Yes. The patterns accept Go / giga and the Arabic word for giga, and prépayé / postpayé; the full-text index folds accents, so “donnees” finds “données”.
- **How does the user know how the query was understood?** Every recognised token is shown as a chip under the search bar, and the user can remove any of them.
- **A plan showed up for “youtube” but I don’t see YouTube on its card — is that a bug?** No. The match is real: one of the plan’s features mentions YouTube, but it may be a line the card normally hides (too long, or Arabic). The results now **highlight that exact feature** on the card, so the reason for every match is visible. (Verified: a search for “youtube” returns exactly the plans whose data genuinely contains it — no false matches.)

Next guide: notifications, or comparison and savings. Tell me which you want.